MyResume – Phase 3: Serverless Infrastructure

Overview

My Resume is an online, cloud-deployed version of my portfolio. This project focuses on the practical application of Microsoft Azure services and demonstrating foundational cloud architecture and deployment skills. This project is developed as the kickstart for my Cloud Computing journey.

Phase 3 of this project involves implementing the serverless version of the project. This includes moving the SQL RDBMS to a NoSQL JSON databse in Azure CosmosDB, creating an Azure Function with a single HTTP trigger to get the Portfolio json data from the database and deploying the React app on Azure Static Web App.

This document will serve as the overview for MyResume Phase 3: Serverless Infrastructure.

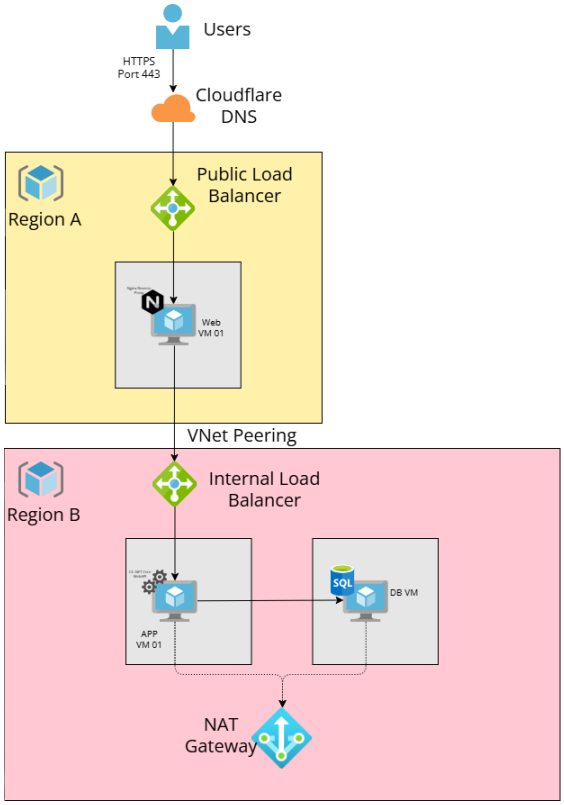
The Tech Stack:

- Frontend: React with Typescript (Deployed over Azure Static Web App)
- Backend: Azure Function
- Database: Azure Cosmos DB

Architecture

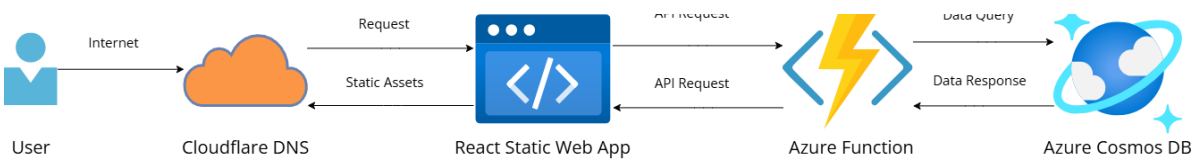
BEFORE - Architecture was a 3-tier structure split into 2 regions with load balancing:

- 2 Separate regions (1 for the frontend & 1 for the backend)
- 1 vm for each tier
- 1 Public & 1 Internal Load Balancer (To mimic actual production environment)



AFTER - This serverless architecture is a simple flow:

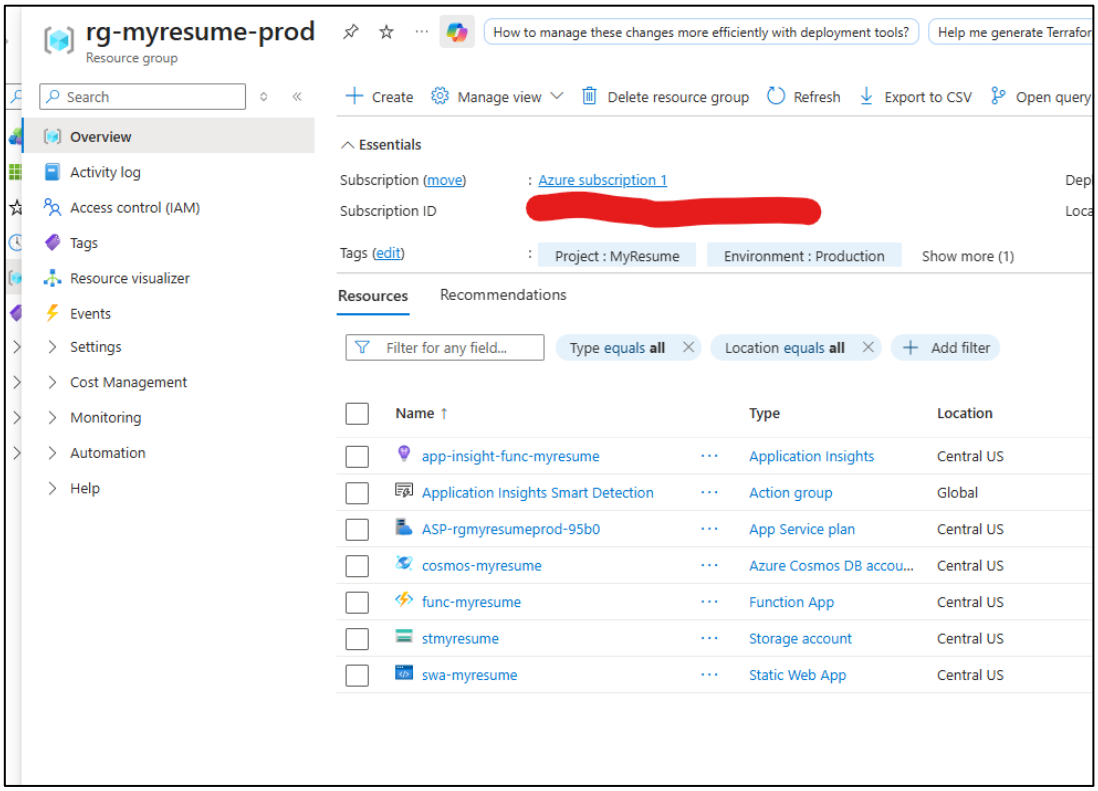
- DNS Resolution & Edge Caching: When a user navigates to the domain, Cloudflare resolves the request and directs traffic to the Azure Static Web App (SWA) origin.
- API Trigger: The React frontend initiates an API request to the Azure Function URL, which executes the serverless backend logic.
- Data Retrieval: The Azure Function establishes a connection to Azure Cosmos DB to perform the requested data query.
- JSON Response: Cosmos DB returns the query results to the Function, which then sends the data back to the React frontend as a structured JSON response.
- Client-Side Rendering: Finally, React processes this JSON data to dynamically update the UI for the user.



Azure Resources

The list of resource provisioned:

- cosmos-myresume (Azure Cosmos DB)
- func-myresume (Azure Function)
- stmyresume (Storage Account to deploy Azure Function)
- app-insight-func-myresume (Application Insight for the function)
- swa-myresume (Azure Static Web App)

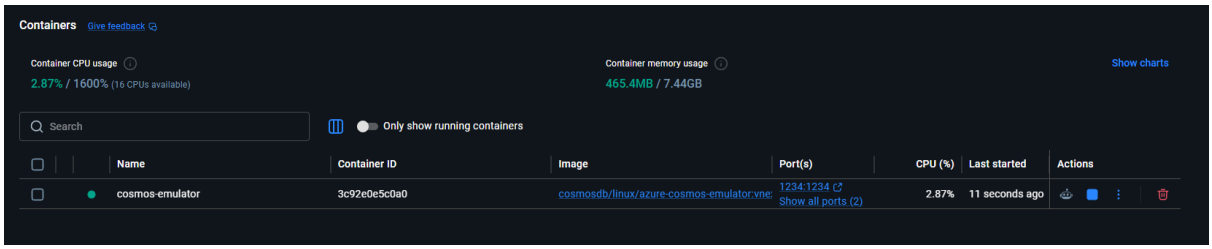


Database

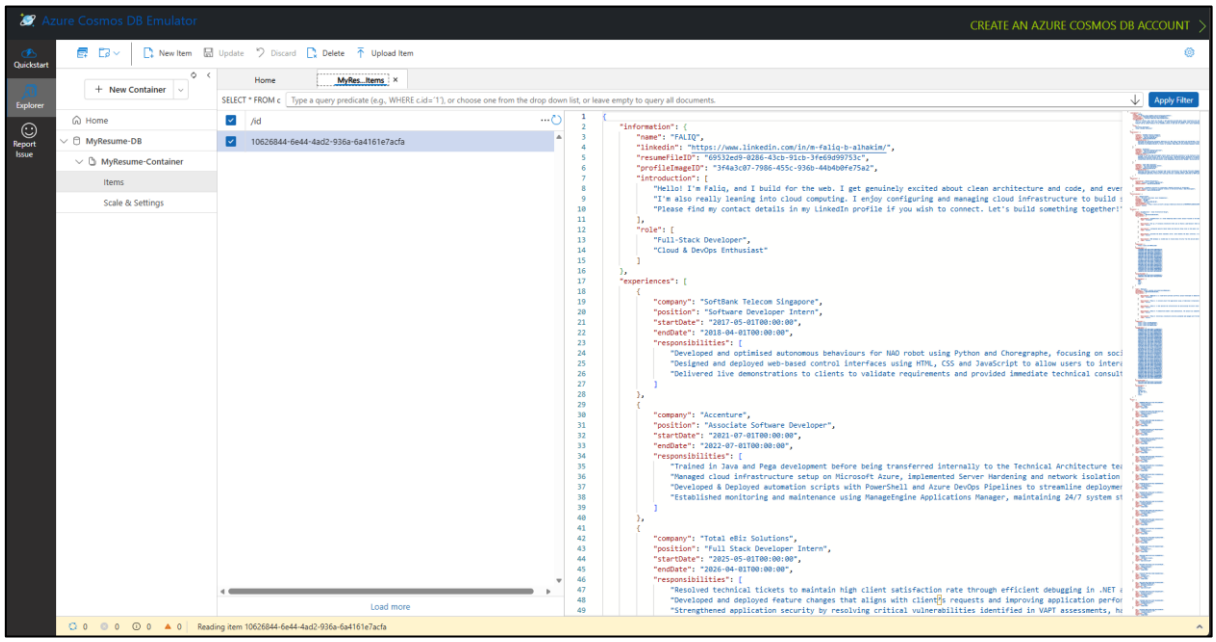
I had to first convert the live data in SQL into a NoSQL JSON file. It was queried from the Portfolio Controller that returned the stitched together data as one JSON response. That response was saved as a JSON file.

An emulation of Azure Cosmos DB was created through a Docker Container. This is a perfect way to test connection and queries locally before moving the live data to the actual Cosmos DB:

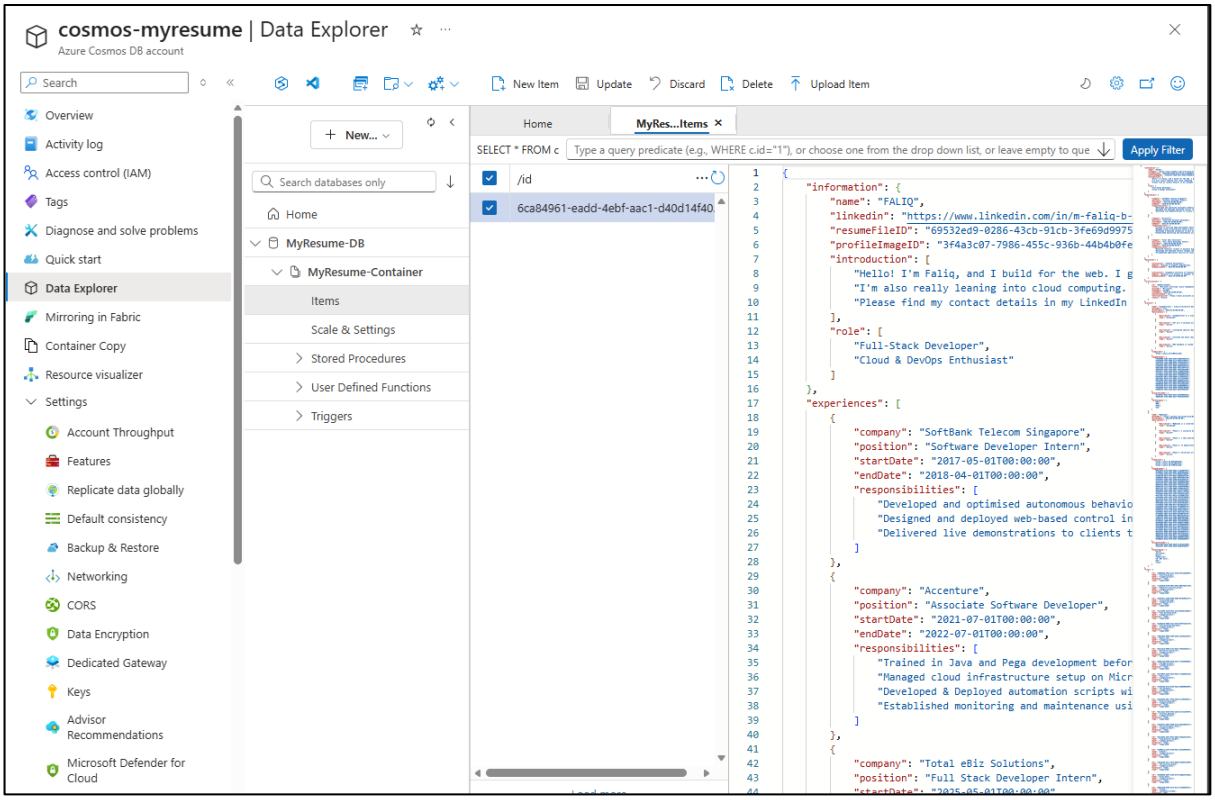
docker run --name cosmos-emulator -p 8081:8081 -p 1234:1234 -d mcr.microsoft.com/cosmosdb/linux/azure-cosmos-emulator:vnext-preview --protocol http



The JSON file is then uploaded as a data entry in database ‘MyResume-DB’ and container ‘My-Resume-Container’.



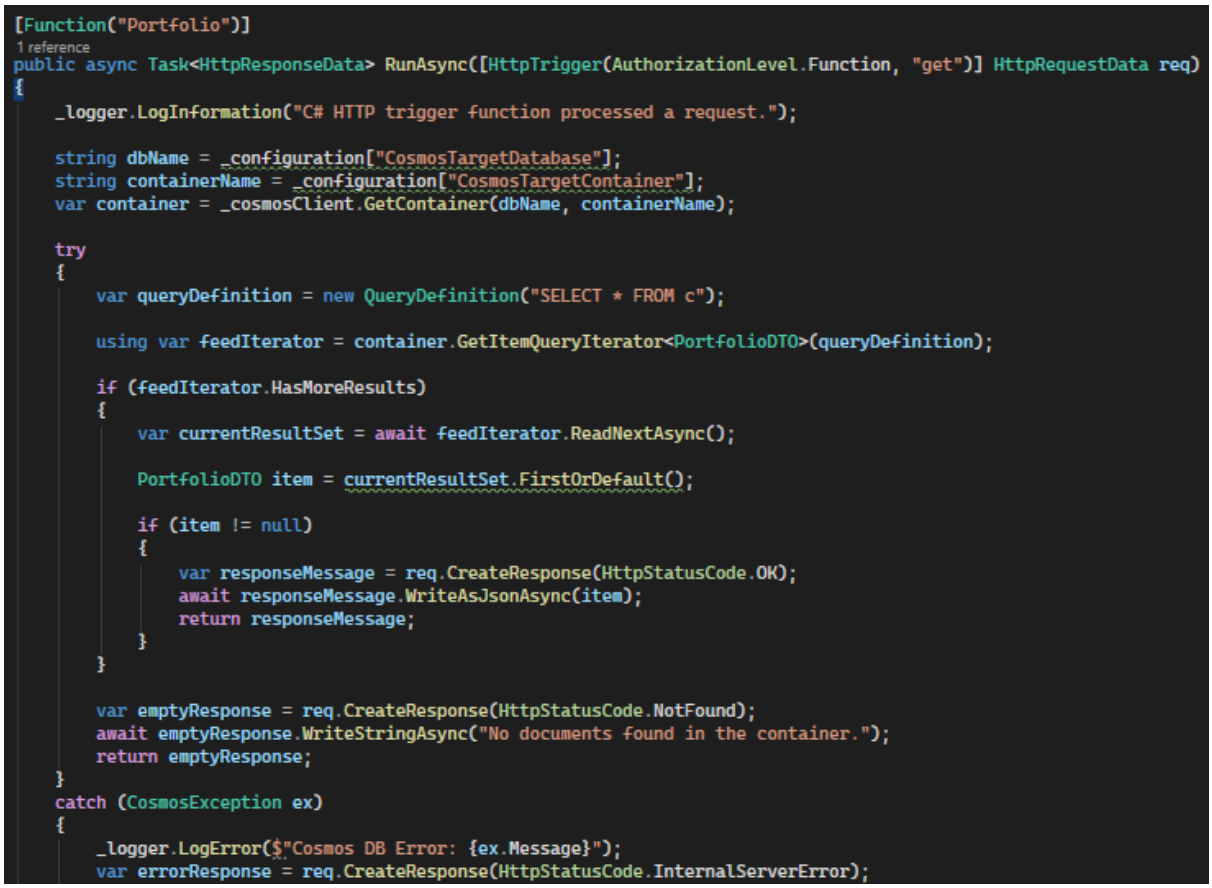
After successful HTTP trigger testing with the implemented Azure Function (Shown later in the document) that returned the data, an Azure Cosmos DB account is created, followed by a Database and Container, similar to the one emulated locally. The JSON file was then uploaded in the same way.



Azure Function

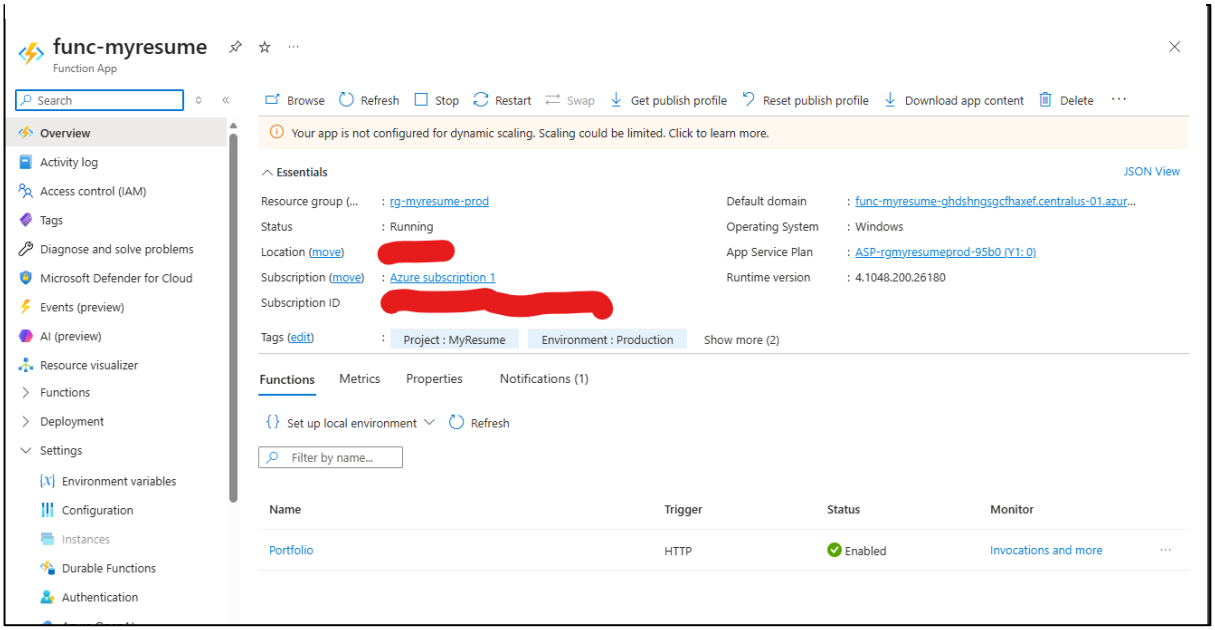
Implementing an Azure Function was a straightforward process, as the core logic closely mirrors a traditional C# .NET Core Web API. Much like a standard Web API, services and database clients are configured in Program.cs, while sensitive configuration keys, such as database connection strings, are managed securely via a local local.settings.json file.

In this implementation, I utilized an HTTP Trigger which acts similarly to the Portfolio Controller in the standard Web API. Given that this is a lightweight portfolio application without complex business logic, I opted to simplify the architecture by omitting the Service and Repository layers and executing the database query directly within the HTTP Trigger. This approach ensures high performance and reduces unnecessary overhead, and the full implementation is available on my GitHub repository.



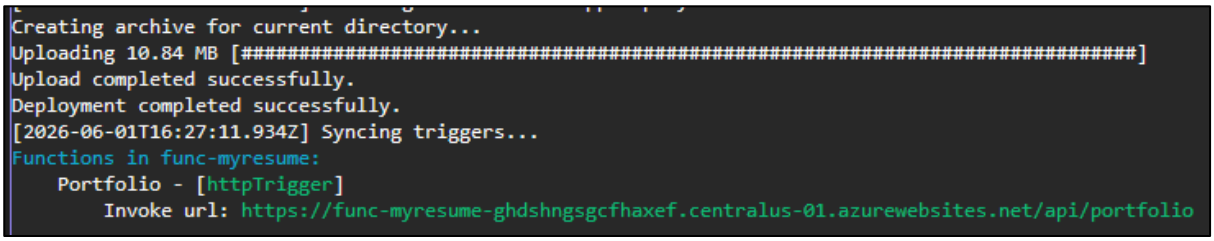
```
[Function("Portfolio")]
1 reference
public async Task<HttpResponseData> RunAsync([HttpTrigger(AuthorizationLevel.Function, "get")] HttpRequestData req)
{
    _logger.LogInformation("C# HTTP trigger function processed a request.");

    string dbName = _configuration["CosmosTargetDatabase"];
    string containerName = _configuration["CosmosTargetContainer"];
    var container = _cosmosClient.GetContainer(dbName, containerName);

    try
    {
        var queryDefinition = new QueryDefinition("SELECT * FROM c");

        using var feedIterator = container.GetItemQueryIterator<PortfolioDTO>(queryDefinition);

        if (feedIterator.HasMoreResults)
        {
            var currentResultSet = await feedIterator.ReadNextAsync();

            PortfolioDTO item = currentResultSet.FirstOrDefault();

            if (item != null)
            {
                var responseMessage = req.CreateResponse(HttpStatusCode.OK);
                await responseMessage.WriteAsJsonAsync(item);
                return responseMessage;
            }
        }

        var emptyResponse = req.CreateResponse(HttpStatusCode.NotFound);
        await emptyResponse.WriteStringAsync("No documents found in the container.");
        return emptyResponse;
    }
    catch (CosmosException ex)
    {
        _logger.LogError($"Cosmos DB Error: {ex.Message}");
        var errorResponse = req.CreateResponse(HttpStatusCode.InternalServerError);
```

An Azure Function, func-myresume, is created in Azure.

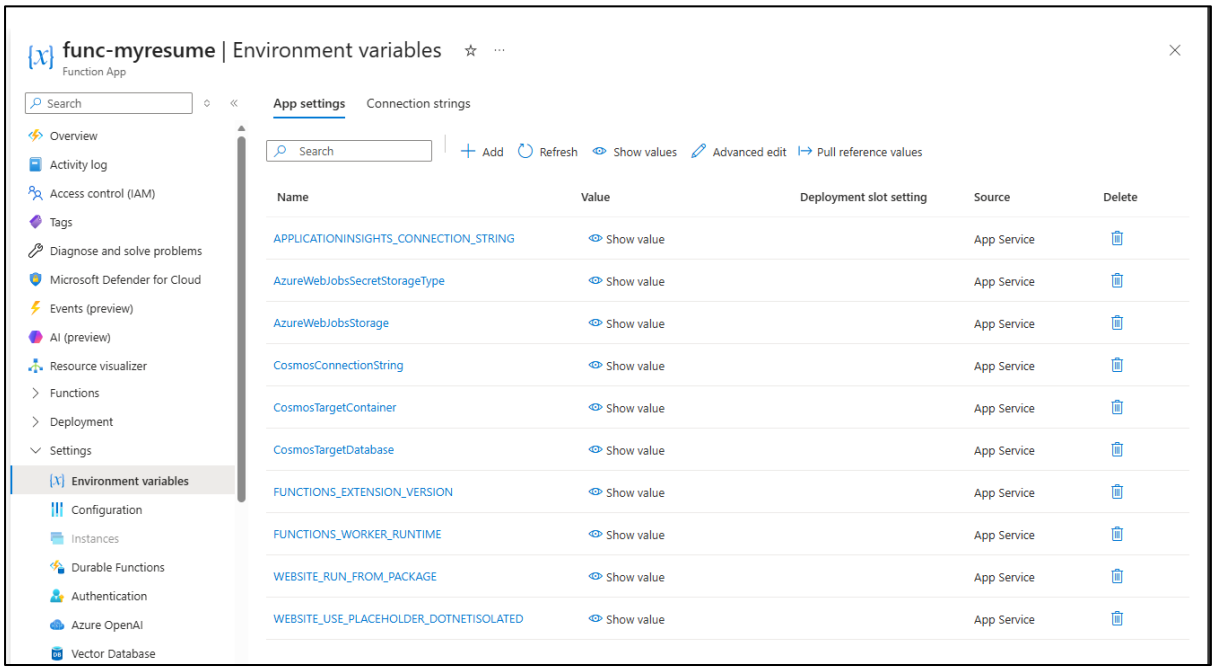


Afterwards, using Azure Function Core Tools CLI, I deployed the Function to the Azure Function in my Azure Portal Account using this command:

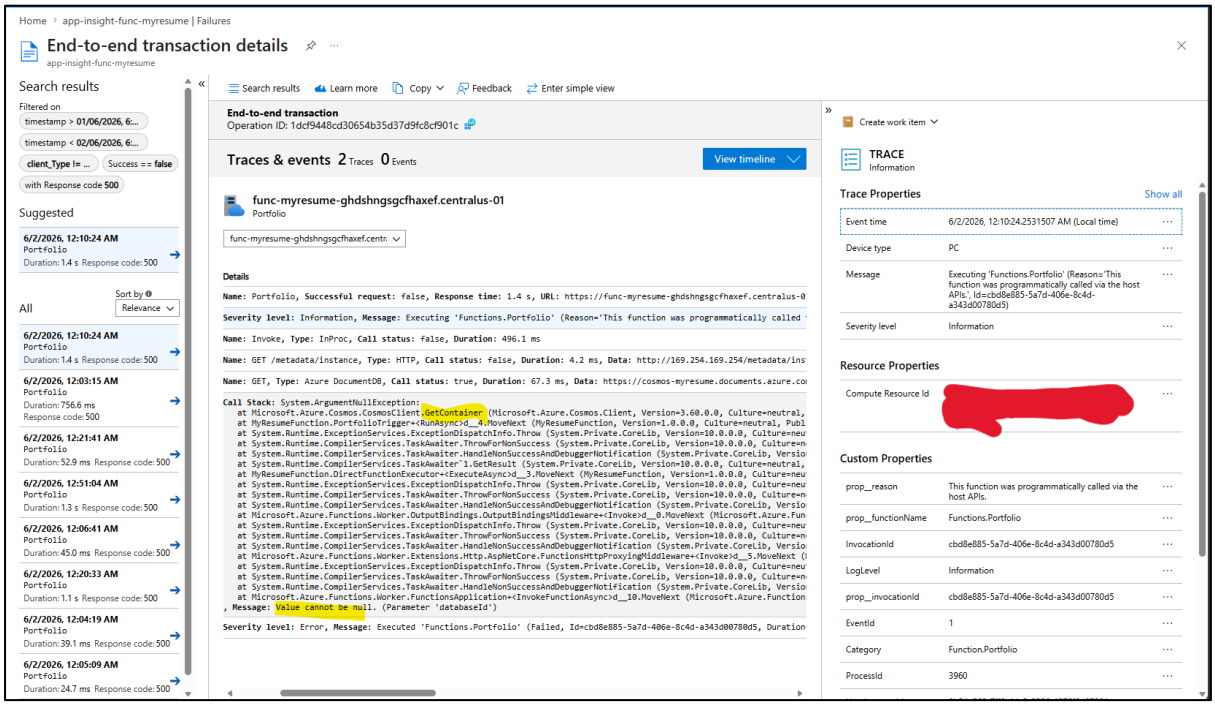
func azure functionapp publish func-myresume



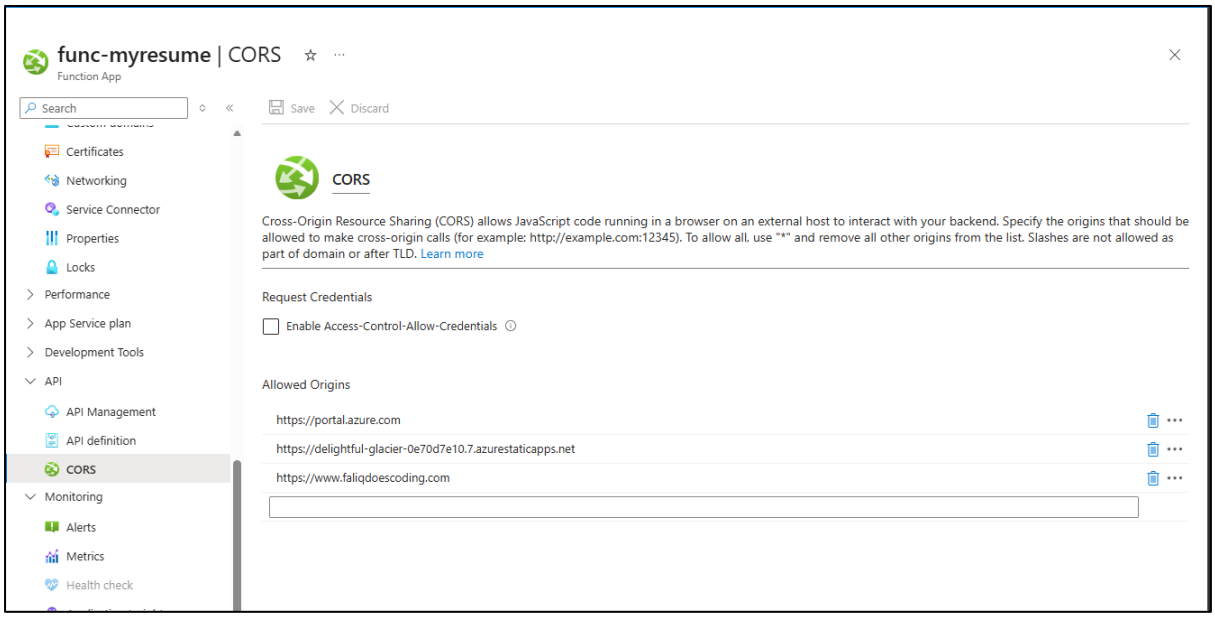
The resource has a setting for Environment Variables, this is variables like db connection string needs to be configured so that the function is able to inject into the code. Notice the two variables ‘CosmosTargetDatabase’ & ‘CosmosTargetContainer’, I was missing them at first and my db query kept failing (Next Image)



In Application Insight, I was able to determine the root cause of the failed db calls because I had forgotten to configure the 2 variables mentioned. After that configuration, the function was able to return the JSON response queried from Cosmos DB.



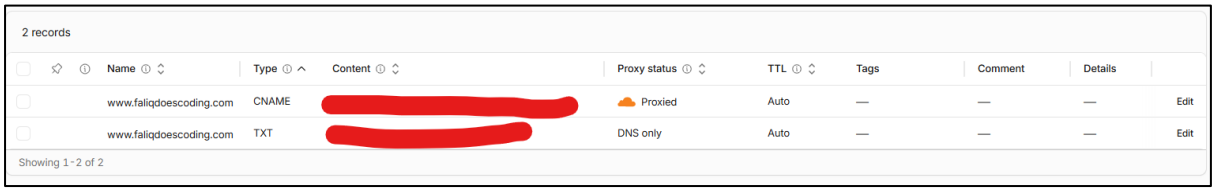
There is another set of setting that needs to be configured which is the Cross-Origin Resource Sharing (CORS). This allows the allowed domains/URLs to perform cross-origin calls (API call/HTTP Trigger) to the function.

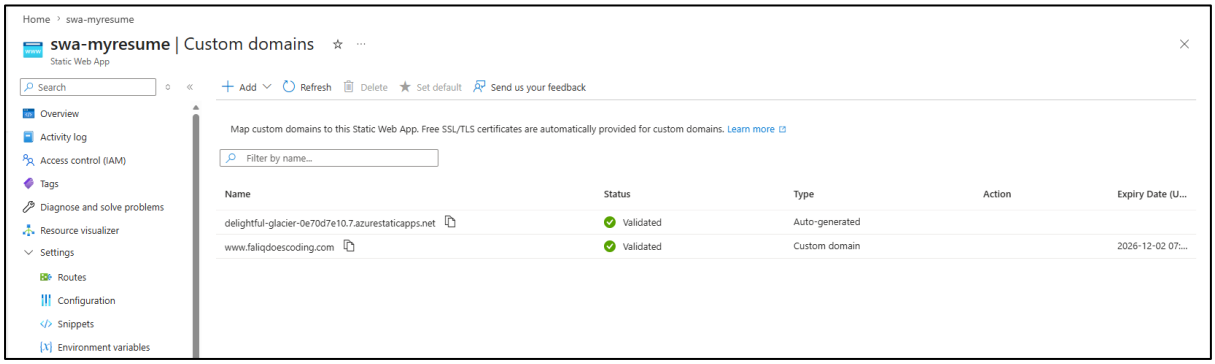


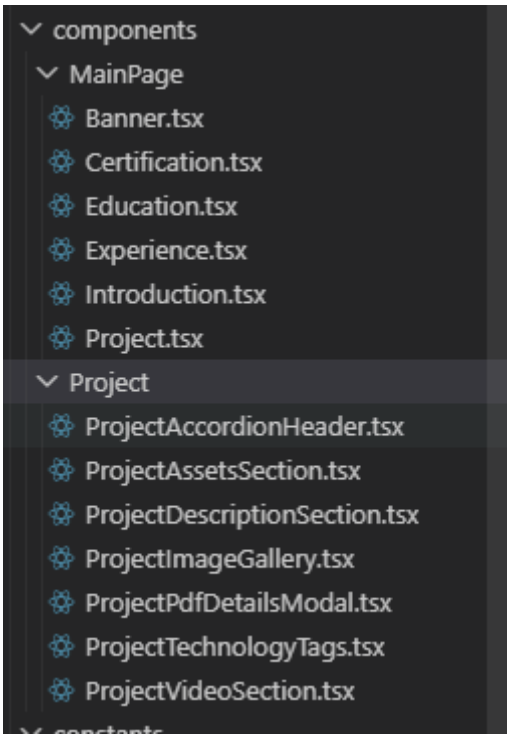
Azure Static Web App

An Azure Static Web App is provisioned in Azure Portal. The frontend code (React) is deployed into this resource through GitHub (I will explain more about this process in ‘Phase 4: Cost-Cutting & Automation’). Simply explained, GitHub will build the React Project and uploads those files to the Azure Static Web App resource.

Afterwards, I had to configure the custom domain from Cloudflare into this resource. Azure has to perform validations that I indeed own the domain by implementing a TXT record in Cloudflare. Once that validation is successful, I can then point the custom domain to the static web app URL.







React code updates:

The Project Component is broken down into more components and the Projects tab is displayed first instead of Experiences.

Technology Tags are also added to each project. A filtering system will be implemented in the future to leverage the tags.